

AI API Testing Platform — Architecture pack

Read this first: what is in the pack, how every page is laid out, and what the colours and tags mean. You do not need the technical specification to use these pages.

What is in the pack (in reading order)		
0	The whole system on one page	who, where the code runs, what it uses, the six rules
—	End-to-end process flow	24 steps, 5 human gates, 5 loops, 1 stop, with release tags
1	Onboard	spec upload, credentials, smoke check, readiness
2	Understand	Discover: the API model · R2 rules from Jira stories
3	Design	rule engine, coverage, review queue, approval · do we need AI?
4	Build	test data, templates, checks, Git
4a	From test case to .feature	one worked example, with and without AI
5	Run	pre-flight, the Kubernetes Job, inside the runner pod, results
6	Close the loop	report · R3 triage, defect review, Jira, rerun on fix
A	How Karate runs on GKE	same command as your laptop, inside a Job
B	How the AI generates test cases	one narrow question, JSON answer, code checks it
C	How test data is made	three sources; AI only reads a CSV's headers
	Appendix	the Python script behind every page, so it can be edited

How every phase page is laid out

title · what the user does · what the platform does · release tags

A the story: 4–5 big steps, left to right, who does each

B the same example followed through every step (small documents)

C callout

C callout

C callout

D a table you will build from

D a table you will build from

E done when · remember

A is the whole phase in one sentence; brackets under the steps say which pod does what.

B is the same real example at every step — read it once and you understand the phase.

C answers the three questions people ask. D is what you will actually write. E is the exit test.

The process flow and overview pages use lanes and rows instead; their legends are on the page.

Colours, tags and shapes		
	white box	code that runs without AI — release 1 unless tagged
	blue box	AI is involved — always release 2 or later; AI only drafts
	green box	a human gate: a person approves, edits, rejects or dismisses
	purple box	release 3: triage, defects, reruns, dashboards
	red box	secrets, or a stop
	amber	a loop, or a status waiting for a person
	grey box	cross-cutting: runs on every request, or deployment notes
MVP	release 1	R2 release 2 R3 release 3
a tag on a box = the release that box ships in; pages say 'build the white parts first'		
◆ diamond = a human gate • dot = start / end dashed box = optional branch		

How to use these pages	
If you are new	read page 0, then the process flow, then the 'step by step' panel of each phase. One hour.
If you are building a phase	read that page's 'build order' and 'done when' panels; the file paths tell you where code goes.
If a word is unclear	every page defines its own words in the last panel; page 0 explains the whole system's words.
If something is missing	a few details are not decided anywhere yet; they are marked on the page ('agree with the QE lead', 'confirm with Advik'). Do not guess silently — raise them.
If a page is wrong	each page is generated by a small Python script (appendix). Change the text in the script, run it, and the SVG and PNG are regenerated. Do not edit the SVG by hand.
What this pack is not	it is not the API contract (that is the generated OpenAPI file) and not the database schema (that is in the migrations). Where this pack and the code disagree, the code wins and the page must be fixed.

The six rules, repeated once here so nobody has to look them up	
1	Deterministic first. Code writes the tests from the spec's facts. AI is asked only for what code cannot know, and it only drafts.
2	A person decides at every gate. Readiness, rules, test cases, data, defects. The platform never files a defect by itself.
3	One test case JSON is the truth. Scripts, Zephyr entries, reports and reruns are all derived from it and reference it by case_id.
4	Secrets never enter the database, logs, Git, prompts or evidence. The only thing stored is the Secret Manager address.
5	Everything is immutable and versioned. Spec versions, suites and runs are never edited; a change is a new version.
6	Every table has team_id and a row-level security policy. The code filters by team; Postgres enforces it a second time.
One image, one core, one database; a folder per phase; a pod group per kind of work. Split into services only if a module later needs its own team, database or release cadence.	

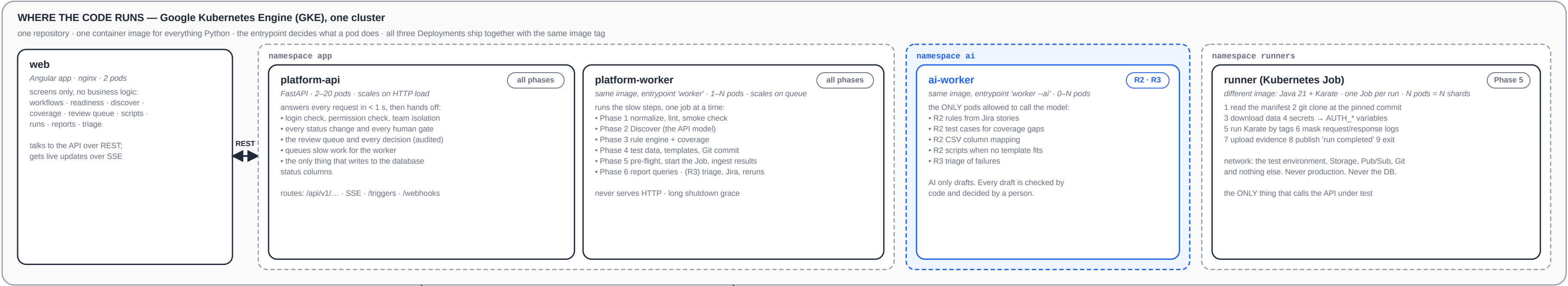
AI API Testing Platform — the whole system on one page

One code base, deployed as three kinds of pods plus a test-runner Job, around one database. People decide at five gates; AI only drafts; the runner is the only thing that touches the API under test.

white = release 1 (MVP) · blue = release 2 (AI) · purple = release 3 (trriage, defects)

read this page first; each phase has its own detailed page

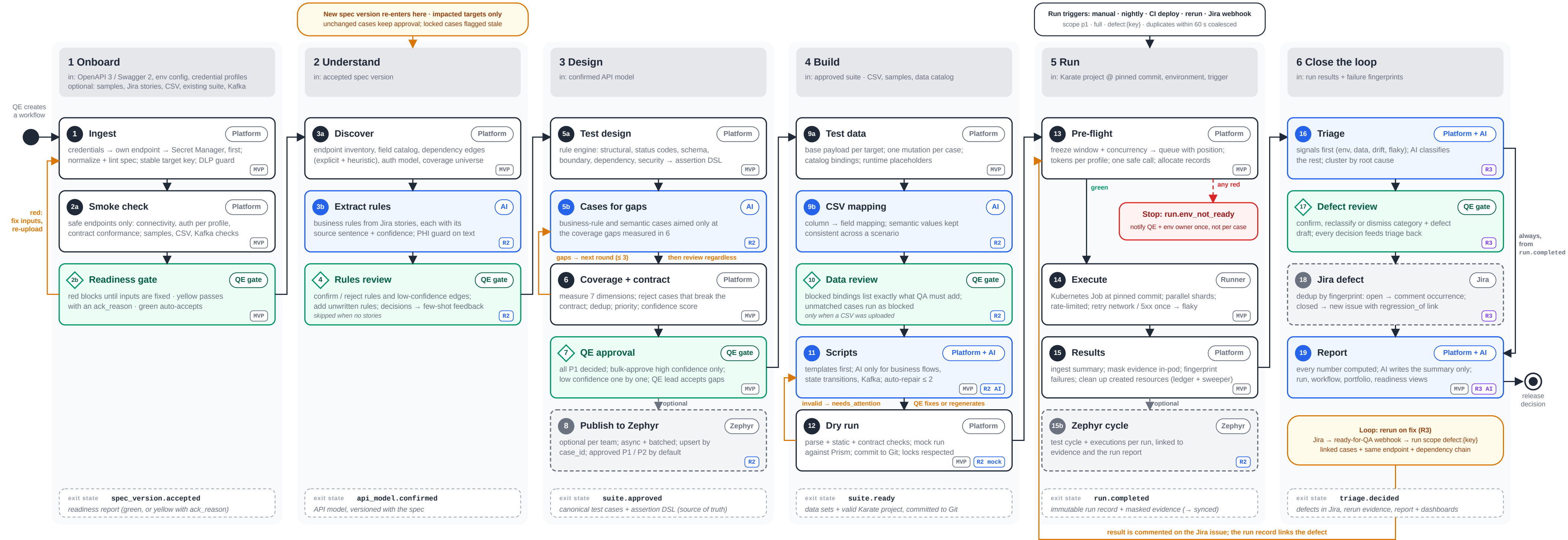
WHO — people and systems that start things



AI API Testing Platform — End-to-end process flow

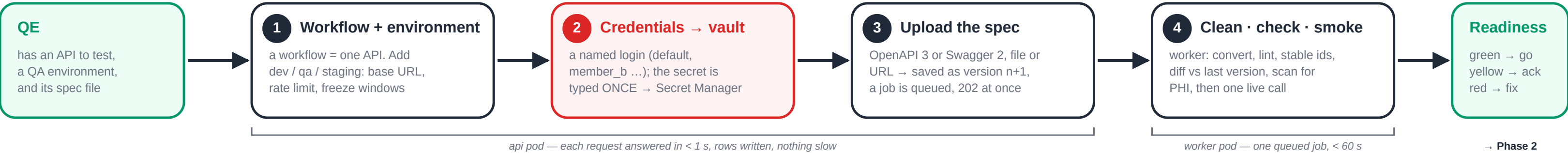
Six phases · 24 steps · 5 human gates · 5 loops · 1 stop. Read left to right; each lane ends in the workflow state that lets the next lane start. AI proposes (blue), the QE decides (green), execution is deterministic.

v4 · aligned to Technical Specification §3 and §8 (24 Sep 2026)
release tags: MVP = no LLM on the critical path · R2 = AI graphs, Jira stories, Zephyr, CSV · R3 = triage, defects, rerun loop

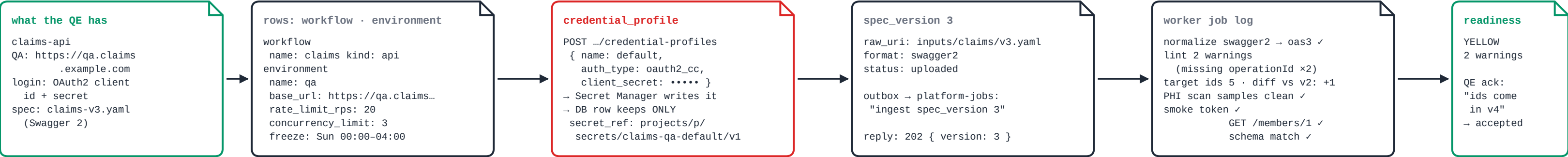


Phase 1 · Onboard

A QE registers one API: where it runs, how to log in, and its OpenAPI spec. The platform locks the secrets away, cleans the spec, and does one live check before anything else may start.



One API followed through every step — claims-api on the QA environment



Secrets: written once, never seen again

The secret arrives in exactly one request body, on its own router whose body is never logged. It goes to Secret Manager and comes back as an address (secret_ref). Only the address is stored. Nothing — not the DB, logs, Git, evidence or a prompt — ever holds the value. Pods that need it read it by address with their own service account. Rotation = new version.

Green · yellow · red — who decides

green: nothing found → accepted automatically, no click.
yellow: warnings only (lint, a sample that does not fit) → the QE presses accept and types a reason; the reason is audited.
red: a blocking failure (cannot reach the host, no token, spec does not parse, PHI in a sample) → nothing can continue. The QE fixes the input and uploads again = a NEW version, never an edit.

api pod vs worker pod — same image, two jobs

The api pod answers every request in under a second: write the row, put a job in the queue (same DB transaction, via the outbox table), reply 202. The worker pod takes the job and does the slow part. Neither knows which it is — the entrypoint decides. This is why an upload never times out and why a deploy never kills a half-done smoke check.

The smoke check — what it does, and what it must never do (make it a package: Phase 5 reuses it)

1 reach	TLS to environment.base_url (client certificate if the profile needs one)
2 log in	a token or key check for EVERY credential profile on the environment
3 one safe call	ONE read-only endpoint per profile; compare the response with the spec's schema
4 samples	if example payloads were uploaded: do they validate against the spec?
result	any blocking failure = red · warnings only = yellow · clean = green, in readiness_report
never	call an endpoint that changes data · touch production · put a secret value in a log or the report

The tables this phase writes — and the one rule on each

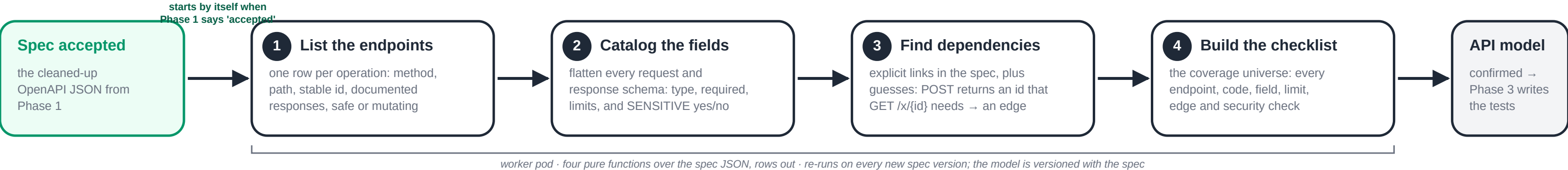
workflow	one API under test; current_spec_version_id points at the latest accepted spec
environment	dev qa staging only; base_url, rate_limit_rps, concurrency_limit, freeze_windows
credential_profile	name, auth_type, scopes, secret_ref — the address, never the value
spec_version	IMMUTABLE: version, format, raw_uri, normalized_uri, lint, diff, readiness, readiness_report, status
audit_event · outbox	who did what (accept reasons live here) · the queued job, written in the same transaction
every table	team_id + a row-level security policy; another team gets 404, in the API and in SQL

Done when: create a workflow, a QA environment and two credential profiles — the secret value appears nowhere · upload an OpenAPI 3 and a Swagger 2 spec, both convert, readiness green in < 60 s · re-upload: the diff lists added / removed / changed · another team sees 404.

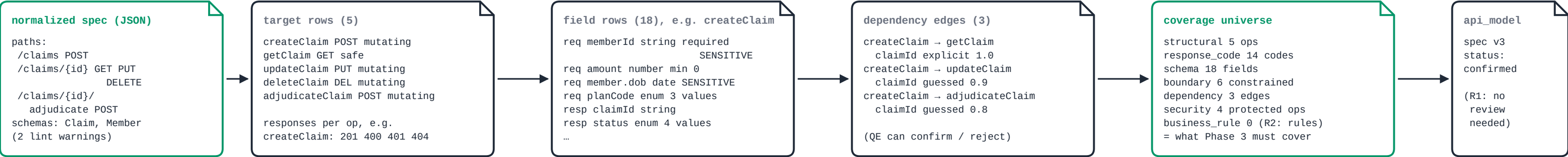
Remember: credentials go to the vault before anything else is stored; a spec version is never edited, a change is a new version; the smoke check reads, it never writes.

Phase 2 · Understand

Read the accepted spec and build the API model: every endpoint, every field, every dependency, and the full checklist of what tests must cover. Automatic, deterministic, no AI in release 1.



One spec followed through every step — claims-api v3 (5 operations)



Sensitive fields are decided HERE — once

Step 2 marks a field sensitive from its name and format (dob, ssn, email, memberId, address ...). A QE can add or remove the flag. From then on, everywhere: the runner masks it in evidence, the log scrubber drops it, and no AI prompt may ever contain its value. Get this list right early — it is the one thing in this phase that a mistake makes expensive.

Guessing dependencies — the rules, and the QE's veto

explicit (confidence 1.0): the spec links them (x-depends, examples)
guessed 0.9: POST /x returns id; GET|PUT|DELETE /x/{id} takes id
guessed 0.8: same field name in a response and another path/body
guessed 0.6: same resource name in the path, no shared field
Only edges ≥ 0.8 or confirmed by a QE count for coverage.
The list of rules is a starting point — agree it with the QE lead.

Release 2: business rules from Jira stories

A QE attaches stories. ai-worker sees ONLY the masked story text, the endpoint names and the field names it mentions. It must return rules with the exact source sentence and a confidence; anything else is rejected before it is saved. The QE confirms, edits or rejects each rule (a gate); only confirmed rules reach Phase 3. Without stories, no gate.

The four tables this phase writes (all keyed by spec_version_id — a new spec version = new rows)

target	one operation: target_key, method, path, safety, auth_scopes, responses, request_schema
field	one field: target, direction req resp, path a.b[]..c, type, required, min/max, enum, sensitive
dependency_edge	from_target → to_target, field_mapping, source explicit heuristic ai, confidence, status
business_rule (R2)	rule_text, applies_to, source_story, source_sentence, confidence, status proposed confirmed
read by	Phase 3 (rule engine, coverage universe) · Phase 5 (masking uses field.sensitive)
shown by	GET /spec-versions/{v}/targets · .../fields · .../dependency-edges · .../coverage-universe

The stable endpoint id (target_key) — why it matters and how it is made

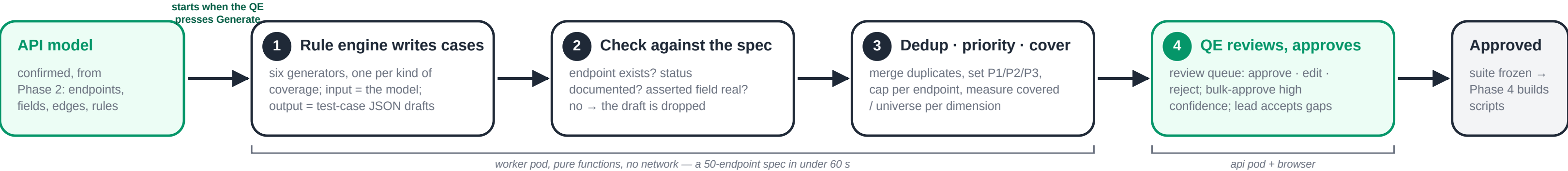
rule	operationId if present and unique; otherwise METHOD + path with every parameter as {}
example	createClaim · or 'PUT /claims/{}' when the spec has no operationId
used by	every test case (coverage_targets), every script (@endpoint tag), every result, every fingerprint
on re-upload	same key = same endpoint: approved cases carry over; a changed schema puts them back to draft
rename detection	different key, identical request + response schema → proposed as a rename, QE confirms
never	changes once a suite is approved — that is what makes history and diffs possible

Done when: open a spec version and see every endpoint with id, safety and responses · every field with type, limits and the sensitive flag · edges with confidence, confirm or reject one · the universe per dimension.

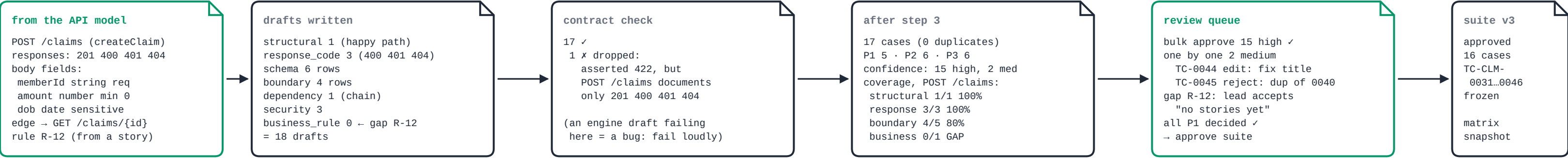
Remember: same spec in, same model out — Discover is a function, not a judgement. The only human decisions here are the sensitive flags and the guessed edges.

Phase 3 · Design

Write the test cases from the facts in the spec, prove each one against the spec, measure coverage, and let the QE approve. No AI in release 1; in release 2 AI fills only the gaps.



One endpoint followed through every step — POST /claims (create a claim)



Do we need AI to write tests? Not in release 1.

Six of the seven kinds of coverage are FACTS in the spec: which endpoints, which status codes, which fields and limits, which calls depend on which, who may call what. Code writes those exactly, every time, and can be unit-tested. The seventh — business rules — lives in prose in Jira stories. Only that needs AI, and that is release 2.

Approve is guarded, not a button

The API refuses to approve unless: every P1 case has a decision, AND every dimension is at its threshold or a QE lead has accepted the gap with a reason. Low-confidence cases can never be bulk-approved. Every edit quotes the row_version it saw; a stale edit gets 409, never a silent overwrite. On approval the suite freezes.

Where AI fits in release 2: the gaps only

After step 3, each endpoint with a gap becomes one AI job. The model sees that endpoint's slice, its documented responses, the confirmed rules, the gap list and 3 approved examples. It returns test-case JSON that goes through the SAME step 2. At most 3 rounds, then review regardless. AI cases are never 'high' confidence, so a person always looks at them.

The six generators — what each one writes from the model alone

structural	one happy path per endpoint: smallest valid body, documented success status
response_code	one case per documented error: 400 drop a field · 401 no login · 403 wrong role · 404 bad id
schema	rows: each required field removed / null / wrong type; extra field if not allowed
boundary	rows: min-1, min, max, max+1, empty, too long, bad enum, bad format
dependency	per confirmed edge: create → get → update → delete → get again (404)
security	no token · expired · wrong scope · another user's record · injection strings (never 500)

One test case = one JSON document (the truth everything else is derived from)

case_id · title · type · priority	TC-CLM-0031 · Reject claim without member id · negative · p1
steps[]	alias, endpoint, login profile, request (body base + ONE mutation), expect (status, assertions)
coverage_targets[]	what this case proves: op:createClaim:400 · field:createClaim:req/memberId
confidence · origin	high = from an explicit spec fact · medium = from a guess or AI · rule_engine ai manual
cleanup[]	which created things to delete after the run
row_version · locked	for safe concurrent edits · a QE-edited case the generator must not touch

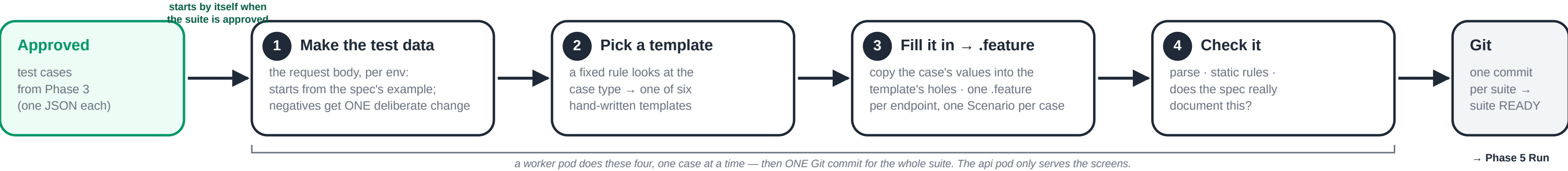
Done when: generate → cases for all six kinds; every case passes the spec check; matrix meets every threshold except business rules · 50 endpoints in < 60 s · bulk-approve high, 409 on a stale edit, approve blocked until P1 reviewed, lead accepts a gap, suite freezes.

Remember: the engine never guesses — no case for a status or a field the spec does not document. A case has ONE deliberate mutation, so a failure has one cause.

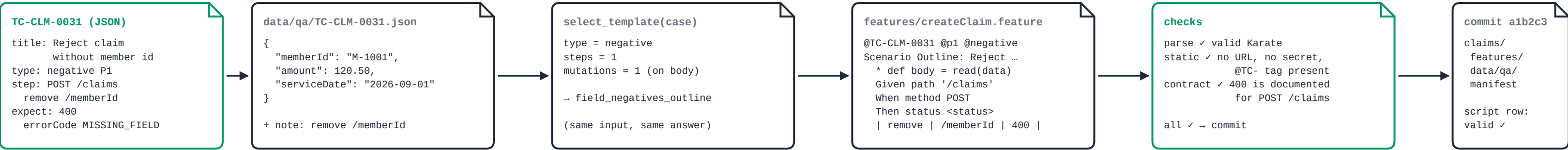
Phase 4 · Build

Turn every approved test case into a Karate script, check it, and put it in Git. Nothing here touches the real API.

release 1 · no AI needed
who: a worker pod does steps 1–4 · the api pod only serves the screens · Git holds the result



The same test case, followed through every step



If a check fails → needs attention

The script is saved but marked 'needs attention'.
A QE opens it in the editor, fixes it, saves.
Nothing is silently dropped; nothing is silently kept.

Screens: Scripts list → Script editor
API: GET /suites/{id}/scripts · GET/PUT /scripts/{id}

A QE edited a script? It is locked.

Saving in the editor commits under the QE's name and sets locked = true.
The generator never overwrites a locked file again; if the spec changes later it only marks it 'stale'.

API: PUT /scripts/{id} · POST /scripts/{id}/unlock

Where does AI come in? Only when no template fits.

Release 2. A multi-step business-rule case has no template, so select_template returns None → ai-worker writes the Scenario, using our helpers and two examples.
It goes through the SAME checks in step 4.
The platform, not the AI, stamps the case_id, tags and URL.
Release 1: such a case is simply 'needs attention'.

The six templates — written by hand first, against the pilot API, before any generator exists

positive_schema_match	happy path: call, expect the status, compare the body to the schema
status_only	401 / 403 / 404 / 405: switch login profile, expect the status
field_negatives_outline	one Scenario Outline, one row per bad field value
dependency_chain	create → get → update → delete, values passed along, cleanup
security_identity	call with another user's login, expect 403 or 404
injection_outline	one row per injection string, expect 400 or 422, never 500

Step 4, the static rules — a script fails if it contains...

- ✗ a URL typed in (it must come from karate-config.js)
- ✗ anything that looks like a secret, or an Authorization header (the runner injects logins)
- ✗ no @TC- tag (the runner picks scenarios by tag, nothing else)
- ✗ anything that looks like personal data
- ✗ a variable used before any step defined it

and the contract rule: every status and field the script asserts must exist in the spec.

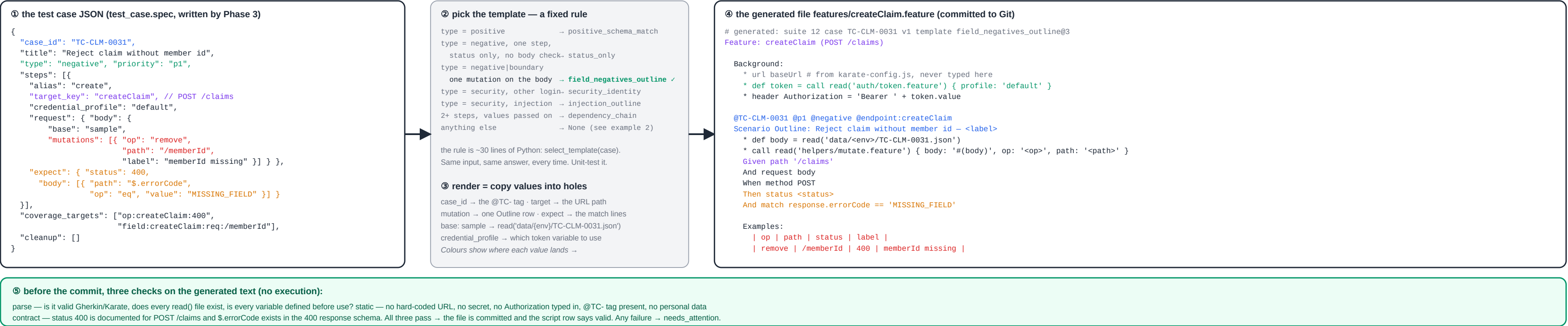
Done when: approve a suite → a Karate project appears in Git with every script valid · edit one script, regenerate, and it is untouched · a run (Phase 5) can start from the commit id.

Remember: the test-case JSON is the truth; a .feature file is just a rendering of it. Fix the template, not the file — unless a QE deliberately locked it.

Phase 4a · How a test case becomes a .feature file — one worked example

A test case is a small JSON document (Phase 3). Build turns it into a Karate script in three moves: pick a template from the case's type → copy the case's values into the template's holes → check the result. No AI is involved for the six standard shapes. AI is asked only when no template fits, and even then it must write inside the same project and pass the same checks.

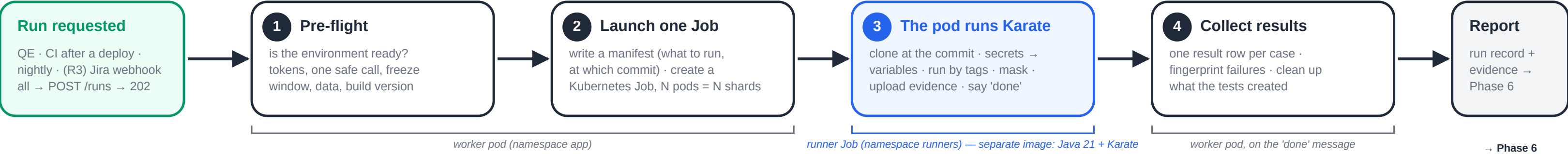
Example 1 — a negative case (release 1, template only)



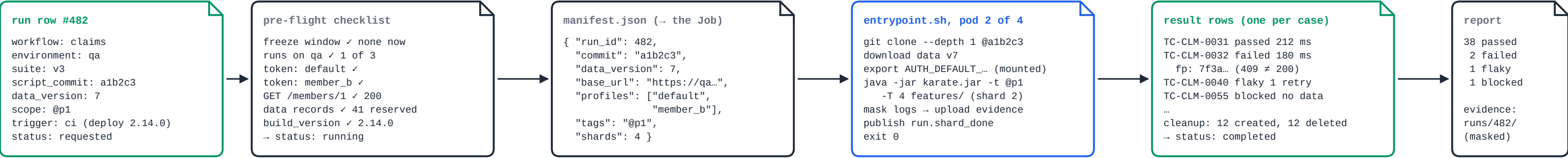
Phase 5 · Run

release 1 · no AI

who: a worker pod does steps 1, 2 and 4 · a runner Job does step 3 · the api pod only takes the request and serves results
Check the environment, start one Kubernetes Job that runs the Karate scripts against it, bring the results back. The runner is the only thing that ever calls the API under test.



The same run, followed through every step — run #482, suite v3, QA environment, scope @p1



Pre-flight red? Stop before spending a Job.

Any red check → status env_not_ready. The run row keeps which check failed. ONE message to the QE and the environment owner — never one per case.

The QE fixes the environment and starts a new run.

Too many runs already? → wait in a queue, position shown.

Cancel at any time: the Job is deleted, reserved data is released, the run is marked cancelled — never half-done.

Flaky vs failed — what gets retried

A scenario whose failure looks like the NETWORK (timeout, connection reset, 5xx) is retried once, inside the pod.

Passes on the retry → recorded as 'flaky', not 'passed'.

An ASSERTION failure (wrong status, wrong field) is never retried: it is the answer. The Job itself never retries.

Same failure text, ids stripped → same fingerprint → Phase 6.

Why the runner is walled off

It never talks to the database: a manifest in, files out.

Secrets are mounted from Secret Manager at pod start and become AUTH_* variables — never in the Job spec, never logged.

Its network reaches the test environment, Storage, Pub/Sub and Git. Nothing else. Production is unreachable by rule.

Masking happens inside the pod, before anything is written.

The Job spec — the lines that matter (deploy/helm/runners/job.yaml, filled in by the launcher)

image	apitest-runner@sha256:... — pinned by digest, scanned in CI
parallelism / completions	= number of shards (max 8); each pod takes JOB_COMPLETION_INDEX
backoffLimit: 0 + podFailurePolicy	never retry on a test exit code; retry ONLY if the node was preempted (spot)
activeDeadlineSeconds	3600 by default (team setting); over it → run.failed
serviceAccount · nodeSelector	runner-sa (may read target secrets, nothing else) · pool=runners, spot allowed
volumes	Secret Manager CSI mount for the profiles in the manifest — no env: secrets

Evidence — what the pod writes to gs://evidence/{team}/{run_id}/, and what is masked first

karate-summary.json	the results the worker reads back; @TC- tag → test case
junit/*.xml · html/	standard reports, for CI and for people
calls/{case_id}/{step}.json	every request and response — this is what a QE opens
logs/pod-{n}.log	the pod's own log
masked before writing	Authorization · Cookie · X-API-Key · every field marked sensitive in the field catalog
kept	full evidence 90 days · summaries 2 years · served via short-lived signed links

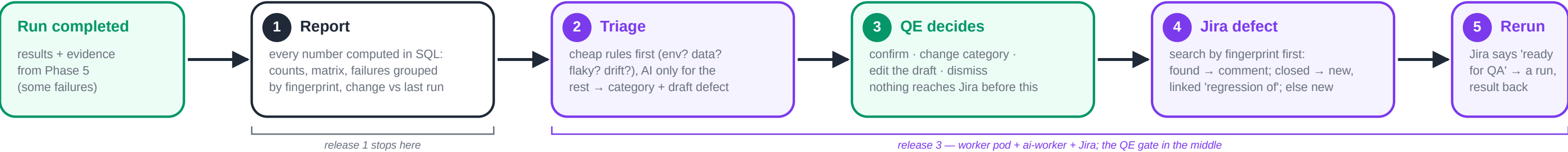
Done when: run on QA → Job starts at the pinned commit, results per case, a canary secret and a sensitive field never appear in evidence, created data is cleaned up · a dead environment → env_not_ready, one message · the run row alone can start the identical run again.

Remember: a run is a Git commit + a data version + an environment + a tag expression. Pin all four and any run can be repeated exactly, next week or next year.

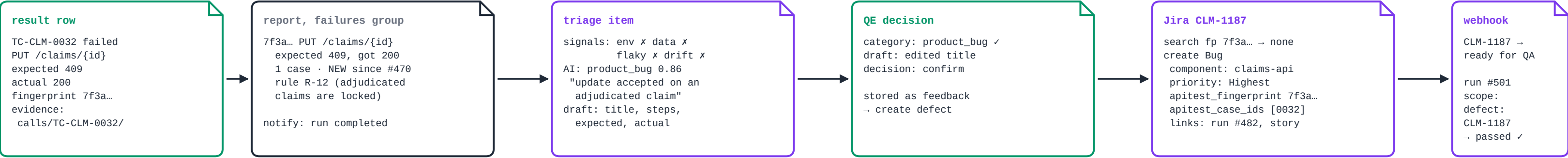
Phase 6 · Close the loop

Turn a failed test into a fixed defect: report it, sort it, let a person decide, file it in Jira once, and re-run the right tests when the fix arrives.

release 1 = the report only · release 3 = everything after it
who: a worker pod does 1, 2, 4, 5 · an ai-worker pod helps in 2 · a QE decides at 3 · Jira is the other end of 4 and 5



The same failure, followed through every step — run #482, case TC-CLM-0032 (update of an adjudicated claim was accepted)



Cheap rules before AI

Four signals settle most failures with no model call:
every case on that host failed → environment issue
the case was blocked / its record stale → data issue
passed on the retry → flaky
response shape changed vs the spec → contract drift
Only what is left goes to the AI, with the masked request/response, the rule and the case's history.

The platform never files a defect on its own

The AI writes a DRAFT; a QE confirms, corrects or dismisses; the worker files it. Every decision is stored — reclassified items become the examples that tune triage later.
One bug = one issue: the fingerprint lives in a Jira custom field, so a repeat is a comment, not a duplicate. Our comments carry a hidden marker so the same event is never posted twice.

Rerun on fix — how the webhook works

Jira calls POST /webhooks/jira when an issue changes status.
The call is verified with a shared secret; the handler reads ONLY the issue key and the new status, nothing else in the body.
Key → our defect row → a Phase 5 run scoped to: the linked cases + every case on that endpoint + its dependency chain.
Passed → comment and close. Failed → comment, stays open.

Triage categories — what each one means and who it goes to	
product_bug	the API did the wrong thing → Jira defect (after the QE confirms)
contract_drift	the API changed and the spec did not → tell the API owner, update the spec
test_issue	our case or script is wrong → back to Phase 3 / 4, no Jira
data_issue	the record we used was missing or stale → fix data, rerun
environment_issue	host down, auth broken → environment owner, rerun
flaky	passed on retry → watch it; three in a row → test_issue

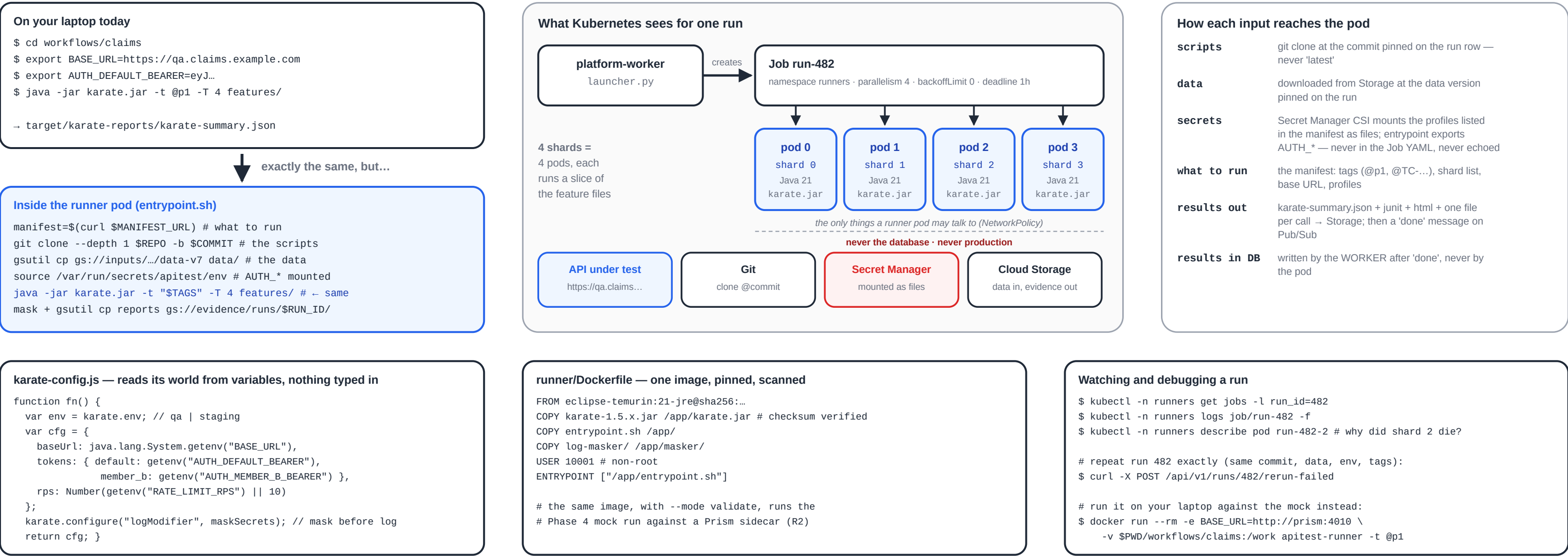
What the Jira issue carries, so it can be found and traced later	
issue type · component	Bug · the API (workflow) it belongs to
priority	from a severity matrix: P1 product bug → Highest
labels	apitest · {workflow_slug}
apitest_fingerprint	custom field — the dedup key; searched before every create
apitest_case_ids	custom field — the TC- ids, so the rerun knows what to run
links	relates to the story · remote links to the run report and the evidence

Done when: a defect on the pilot API goes failed test → triage → confirmed → Jira → fix → 'ready for QA' → automatic rerun → passed, with no manual step except the QE's decision · the same failure in two runs makes ONE issue.

Remember: numbers are computed, never written by AI; AI drafts, a person decides, the worker files. If triage is wrong more than 15 % of the time on the labelled set, it does not ship.

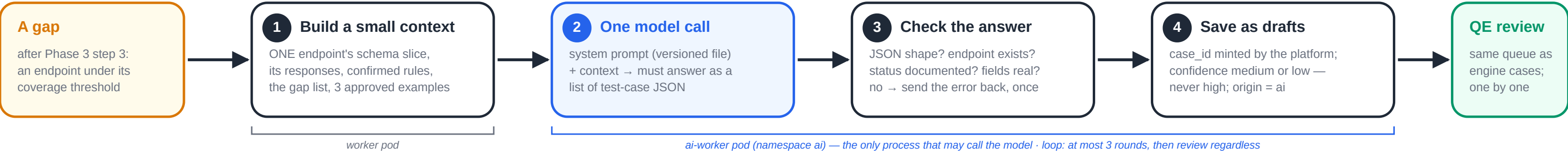
How Karate runs on GKE — for a developer who has only run it on a laptop

Short version: it is the same command you run locally, inside a pod, started by a Kubernetes Job. Everything else on this page is about how the pod gets its inputs and where its outputs go.

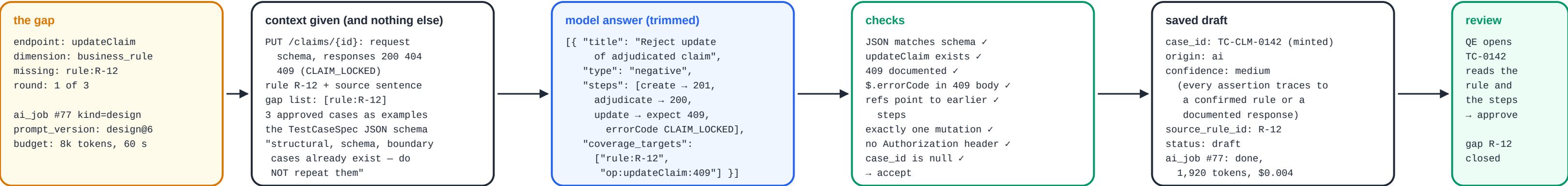


How the AI generates test cases — it is a pipeline with one model call, not a chatbot

Release 2. The rule engine has already written every case the spec's facts allow. The AI is asked one narrow question per endpoint: 'here are the gaps — propose cases for them, as JSON'. Then code checks the answer.



One gap followed through — PUT /claims/{id}, business rule R-12 'an adjudicated claim cannot be updated' is not covered



What the model never sees

the whole spec (only one endpoint's slice)
real data records (only field names and value shapes)
credentials, tokens, hostnames in stack traces
any field marked sensitive — the PHI guard runs on every
prompt before it leaves the platform
The context is small on purpose: cheaper, faster, and the
model cannot invent cases for endpoints it was not shown.

Why an AI case can never be 'high' confidence

high = derived from an explicit fact in the spec by code.
An AI case is a proposal: medium when every assertion traces
to a confirmed rule or a documented response, low otherwise.
Low-confidence cases cannot be bulk-approved — a person reads
each one. This is the whole safety model: code proves, AI
proposes, a QE decides. Nothing invalid is ever saved as a draft.

What we measure before it ships, and the kill switch

A golden set: 3–5 internal APIs with approved cases. On every
change to a prompt, model or graph, CI runs the set and checks:
≥ 95 % of AI cases pass the contract check first try
≥ 90 % business-rule coverage after 3 rounds
≤ 10 % duplicates · cost ≤ 120 % of the previous baseline
Pilot rule: if QEs reject > 30 % of AI drafts, keep AI for rules only.

The graph in code — a fixed sequence, no free-form agent (ai/graphs/design/)

nodes slice_context → call_model → validate → (repair once) → save_drafts — each a plain function
LLMProvider.complete() the ONE way to call a model: prompt, JSON schema, tier fast|strong, token budget, timeout
prompts/ versioned files; prompt_version is stored on every ai_job so a result can be explained later
in tests a recorded-response provider replaces Gemini, so the graph runs in CI without a model

Where else the same pattern is used (same graph shape, different question)

rules (R2) 'here are Jira stories — list the business rules, each with its source sentence'
csv_mapping (R2) 'here are CSV headers + 5 masked rows — map columns to these fields, with confidence'
scripts (R2) 'here is one case with no template — write the Scenario inside this framework'
triage (R3) 'here is a failure the rules could not settle — category, reason, draft defect'

Remember: the model is asked a narrow question with a JSON answer; code decides whether the answer is usable; a person decides whether it is right.

How test data is made — three sources, and AI is only in one of them

Every test case needs a request body and, for chains, real ids. Most of it is built by code from the spec. AI helps in one place: understanding a CSV a QE uploads. Real records come from a catalog, matched by code.

1 · From the spec (release 1, code only)

every case, always

The rule engine knows each field's type, format, limits and example. It builds the smallest valid body, then applies the case's ONE change. Values it must invent follow the spec: a date is a date, an enum is one of the listed values, an id is 'gen: uuid' filled in at run time.

Chains pass ids along: `${create.response.body.claimId}` becomes a Karate variable — no id is ever typed into a file.

example

```
data/qa/TC-CLM-0031.json
{ "memberId": "M-1001", ← from the spec example
  "amount": 120.50, ← example
  "serviceDate": "${today-30d}", ← gen at run time
  "planCode": "GOLD" } ← first enum value
mutation: remove /memberId ← the ONE change
```

2 · From a CSV a QE uploads (R2, AI helps once)

optional, per endpoint

QEs often have real-looking rows in a spreadsheet: member numbers, plans, dates. The AI's job is small: read the column names and 5 MASKED rows and say which field each column feeds, with a confidence. The QE confirms the mapping (a gate that exists only when a CSV was uploaded). After that, code turns rows into data files — no AI.

Low-confidence columns are shown first; unmapped columns are ignored.

example

```
members.csv: member_no, birth_dt, plan, zip
AI mapping:
  member_no → memberId 0.95 ✓
  birth_dt → member.dob 0.98 ✓
  plan → planCode 0.70 ? (QE confirms)
  zip → (none) –
→ data/qa/TC-CLM-0040.csv (one row = one scenario run)
```

3 · From the QA data catalog (R2, code only)

when a case needs a REAL record

Some cases need a record that already exists in the environment: 'an active member over 65'. The case states that as a filter, not a value. The catalog (uploaded by QA, per environment) holds such records; pre-flight matches by filter and RESERVES one for the run.

No match → the case runs as 'blocked' and the report says exactly what QA must add. Consumable records are released after the run.

example

```
case data_requirement:
  { "status": {"eq": "ACTIVE"},
    "age": {"gte": 65} } mode: consumable
catalog qa: 41 members match → reserve 1
           → memberId M-2207 injected at run time
no match → result: blocked
"need: ACTIVE member, age ≥ 65"
```

What the AI never sees, even here

Only the column names and 5 rows, and those rows are masked first (the PHI guard runs on the sample). It never sees the whole CSV, never a real catalog record, never a full data set. It answers one question — which column is which field — and that answer is checked against the schema (types must fit) and confirmed by a QE before a single data file is written from it.

Why 'realistic' values are NOT an AI job in release 1

The spec already says what a value must look like: format, min/max, pattern, enum, example. Code that follows those rules produces valid data every time and never leaks anything. A model asked for 'a realistic member' might produce a real one. If a team wants prettier names later, that is a small, separate R2 feature with the same guard — not part of the core path.

Where the data lives and how it is versioned

`data/{env}/{case_id}.json` — the body, per environment
`data/{env}/{case_id}.csv` — rows for data-driven cases
Both are built by the Phase 4 worker, stored in the inputs bucket and committed with the scripts. A run pins a `data_version`, so a re-run a month later uses the same rows. Catalog records are the exception: matched fresh at pre-flight, then released.

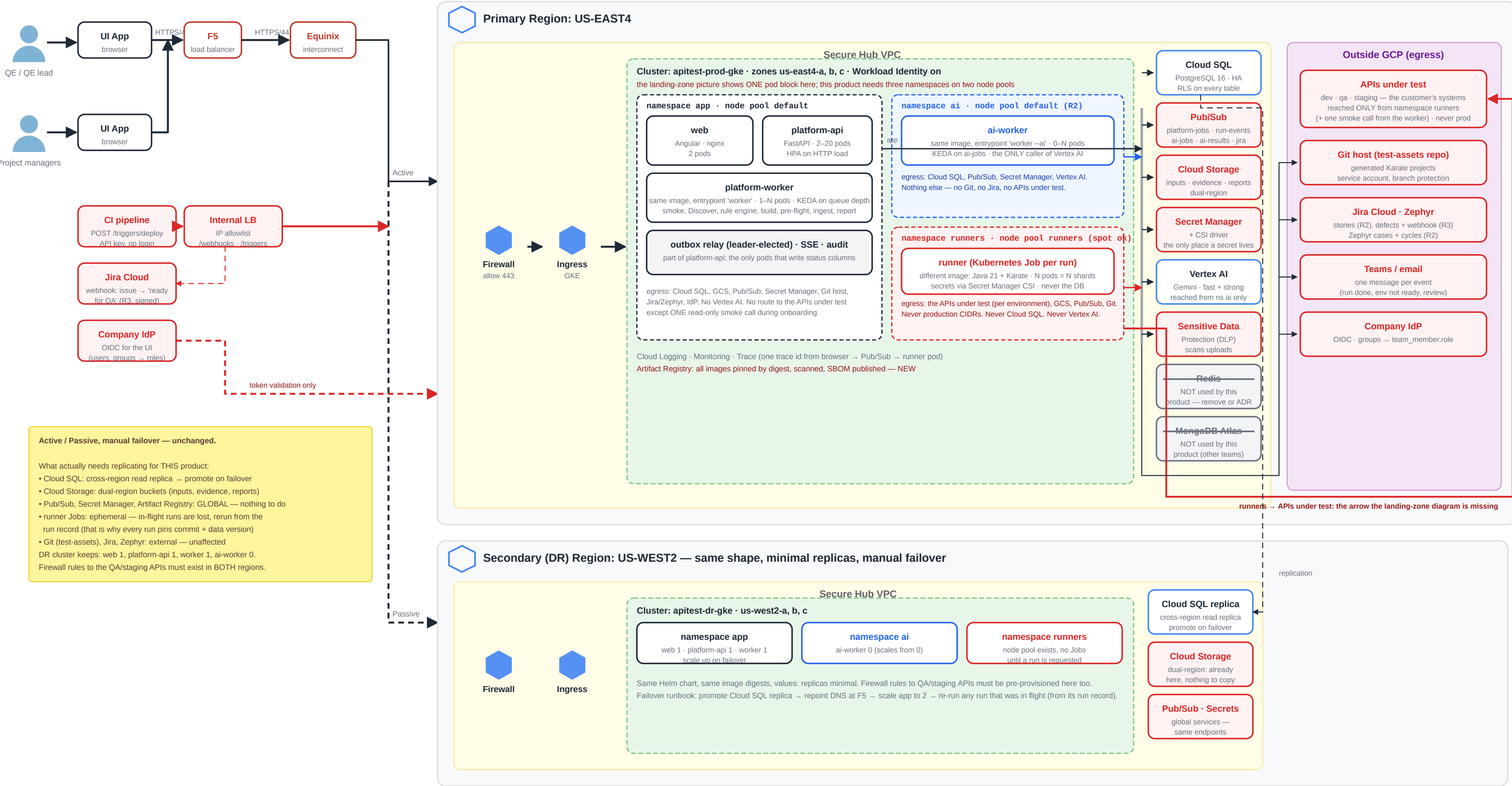
Who does what — the honest split

code, from the spec	builds every body, applies every mutation, generates ids/dates/enums, passes ids along chains — release 1, always
AI, once per CSV	proposes column → field mapping from headers + 5 masked rows; a QE confirms — release 2, only if a CSV exists
code, from the catalog	matches a case's filter to real records, reserves one per run, blocks the case if none — release 2
a QE	uploads the CSV, confirms the mapping, keeps the catalog current, reads the blocked list
never	AI inventing 'real-looking' personal data · AI seeing a full CSV or a catalog record · a real id typed into a file

Remember: the spec is the source of every value the platform invents; a QE is the source of every real one; AI only translates a spreadsheet's column names.

AI API Testing Platform — GCP deployment (same format as the landing-zone diagram)

Primary us-east4 shown in full; the DR region is identical with minimal replicas. Items in RED are new compared with the landing-zone picture; items with a strike are not used by this product.



Legend: blue = already on the landing-zone diagram · RED = add for this product · grey with strike = on the diagram but not used by this product · dashed boxes = namespaces · dashed red arrow = secrets read · purple dashed = DLP scan

Snippet 1 — replace the single 'pod / auto-scaling' block with this

Cluster: apitest-prod-gke · Workload Identity on · one image for everything Python, the entrypoint decides

namespace app · node pool default

web

Angular · nginx · 2 pods

platform-api

FastAPI · 2–20 pods
HPA on HTTP · has the Service + Ingress

platform-worker

same image, entrypoint 'worker' · 1–N pods · KEDA on queue depth
runs every slow step: smoke, Discover, generate, build, pre-flight, ingest

egress: Cloud SQL · Storage · Pub/Sub · Secret Manager · Git · Jira/Zephyr · IdP
NOT Vertex AI · NOT the APIs under test (one read-only smoke call excepted)

namespace ai · node pool default (release 2)

ai-worker

same image, entrypoint 'worker --ai' · 0–N pods · KEDA on ai-jobs
the ONLY pods allowed to call Vertex AI

egress: Cloud SQL · Pub/Sub · Secret Manager · Vertex AI — nothing else

namespace runners · node pool runners (spot allowed)

runner — a Kubernetes Job per test run

different image: Java 21 + Karate · N pods = N shards
secrets mounted (Secret Manager CSI) · never the database

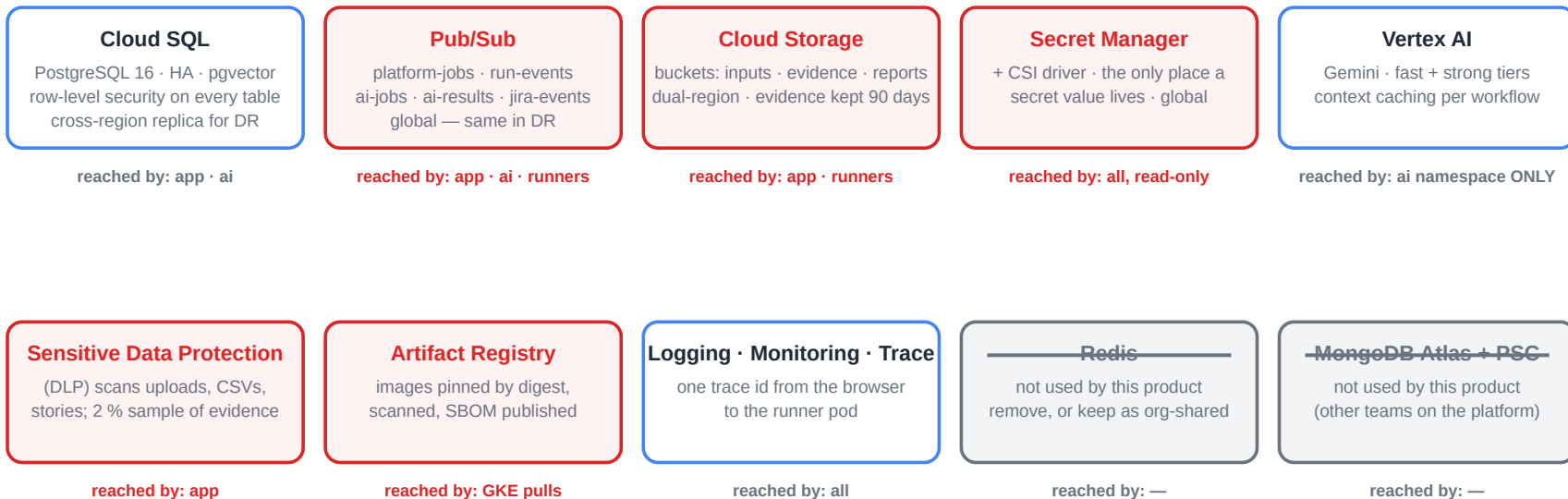
egress: the APIs under test (per environment) · Storage · Pub/Sub · Git

NEVER production CIDRs · NEVER Cloud SQL · NEVER Vertex AI

podFailurePolicy: retry only on node preemption, never on a test exit code

Arrow in from: Ingress → platform-api (and web). Arrows out: see snippets 2 and 3.

Snippet 2 — the managed services column inside the VPC (blue = already on your diagram, red = add)



Draw ONE line from the cluster to each service; the 'reached by' label says which namespace may use it (that is the NetworkPolicy).

Snippet 3 — who calls in, what we call out (all new on your diagram)

CALLS IN (left side of your diagram, next to the users)

CI pipeline

POST /triggers/deploy
team API key, not a user login
→ a P1 run after every deploy

Jira Cloud (webhook, R3)

issue → 'ready for QA'
signed with a shared secret
→ rerun the linked tests

Company IdP (OIDC)

the UI signs users in here;
the API validates the token
groups → QE / QE lead / admin

Internal load balancer

IP allowlist · only /webhooks
and /triggers · NOT via F5

CI and Jira come in through the internal load balancer, not through F5; the IdP is only called by the browser (sign-in) and by the API (token check).

CALLS OUT (right side of your diagram, outside the VPC)

APIs under test

dev · qa · staging
the customer's systems
reached ONLY from namespace runners
(+ 1 smoke call from the worker)

Git host (test-assets)

generated Karate projects
service account, branch protection
cloned by the runner at a pinned commit

Jira · Zephyr

read stories (R2) · write defects (R3)
publish cases + test cycles (R2)
REST from platform-worker only

Teams / email

one message per event with a link
run done · env not ready ·
suite awaiting review

The one arrow that matters most

namespace runners → APIs under test. Draw it in red. It is a firewall rule PER environment (dev, qa, staging) and it must exist in BOTH regions. There is no rule to production, and no other namespace has this route. Everything else on this page is ordinary egress from the app namespace. Put it on the diagram as: runners -(443, per-env allowlist)→ 'APIs under test' box outside the VPC.

Snippet 4 — replace the sticky note with this one

Active / Passive. Primary us-east4, DR us-west2. Manual failover — unchanged.

What is actually regional for THIS product (and therefore needs replicating):

- Cloud SQL — cross-region read replica; promote on failover
- Cloud Storage — make the three buckets dual-region; nothing to copy at failover time
- the GKE cluster — same Helm chart, same image digests, replicas: web 1, api 1, worker 1, ai-worker 0
- firewall rules from namespace runners to the QA / staging APIs — pre-provisioned in BOTH regions

What is global (nothing to do): Pub/Sub · Secret Manager · Artifact Registry · IdP · Logging

What is external (unaffected): Git host · Jira · Zephyr · the APIs under test

Runner Jobs are ephemeral. A run in flight at failover is LOST — and that is fine:

every run pins commit + data version + environment + tags, so it is re-run from its run record.

Failover runbook: 1 promote Cloud SQL replica 2 repoint DNS at F5 3 scale app to normal

4 re-run anything that was in flight 5 Zephyr / Jira need no action (they were never regional).